Generative Adversarial Networks (GANs)
From VAE Motivation to Theory, Training, Limitations, and Conditional GANs

March 13, 2026

Roadmap

1. Warm-up: VAEs and Why GANs Were Introduced
2. GAN Intuition and Setup
3. How to Train GANs (practical algorithm + stability tips)
4. Wrap-up
5. Case Study: Tail-GAN for Tail Risk Simulation
6. Wasserstein GAN (WGAN)

Goal: Learn a Generative Model

We observe $x \sim p_{\text{data}}$ over $\mathcal{X} \subseteq \mathbb{R}^d$ and want a model distribution p_g that can *sample* realistic data.

$$\text{want: } p_g \approx p_{\text{data}} \quad \Longrightarrow \quad x \sim p_g \text{ looks like } x \sim p_{\text{data}}.$$

Two broad families:

- **Likelihood-based** (e.g., VAE): maximize (a lower bound on) $\log p_\theta(x)$.
- **Implicit models** (e.g., GAN): define sampling procedure $x = G_\theta(z)$ without explicit tractable likelihood.

VAE Review: Latent Variable Model

VAE assumes a latent variable model:

$$z \sim p(z) = \mathcal{N}(0, I), \qquad x \sim p_\theta(x|z).$$

Marginal likelihood:

$$p_\theta(x) = \int p_\theta(x|z)p(z)\,dz \quad \text{(usually intractable)}.$$

Introduce an inference network (encoder) $q_\phi(z|x)$.

VAE Objective: ELBO (math)

Start from:

$$\log p_\theta(x) = \log \int p_\theta(x|z)p(z)\,dz = \log \int q_\phi(z|x)\frac{p_\theta(x|z)p(z)}{q_\phi(z|x)}\,dz.$$

Apply Jensen:

$$\log p_\theta(x) \geq \mathbb{E}_{z\sim q_\phi(z|x)}\left[\log p_\theta(x|z)\right] - \mathrm{KL}\big(q_\phi(z|x)\,\|\,p(z)\big) \quad =: \quad \mathcal{L}_{\mathrm{ELBO}}(x).$$

Equivalent decomposition:

$$\log p_\theta(x) = \mathcal{L}_{\mathrm{ELBO}}(x) + \mathrm{KL}\big(q_\phi(z|x)\,\|\,p_\theta(z|x)\big).$$

VAE: Reparameterization Trick (math)

If $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}(\sigma_\phi(x)^2))$, sample via

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon,$$

so gradients flow through z:

$$\nabla_\phi \mathbb{E}_{z \sim q_\phi(z|x)}[f(z)] = \nabla_\phi \mathbb{E}_{\epsilon \sim \mathcal{N}(0,I)}[f(\mu_\phi(x) + \sigma_\phi(x) \odot \epsilon)].$$

Why GANs? (motivation vs likelihood training)

Common practical tensions (esp. early VAEs in images):

- Likelihood objective often implies *pixel-wise* losses (e.g. Gaussian decoder $\Rightarrow$ MSE) $\Rightarrow$ blurry samples.
- You optimize a bound on $\log p_\theta(x)$, not always aligned with perceptual realism.
- Implicit models: optimize a *distribution match* via a learned critic/classifier rather than explicit likelihood.

GAN idea: learn p_g by **adversarial training** against a discriminator that learns a moving notion of realism.

GAN Ingredients

- Noise prior: $z \sim p_z$ (e.g. $\mathcal{N}(0, I)$).
- Generator: $G_\theta : \mathcal{Z} \to \mathcal{X}$, producing $\tilde{x} = G_\theta(z)$, hence $\tilde{x} \sim p_g$.
- Discriminator: $D_\psi : \mathcal{X} \to (0, 1)$, interpreted as $D_\psi(x) \approx P(\text{real} \mid x)$.

Adversarial game: D distinguishes real vs fake, G makes fakes indistinguishable.

The Classic GAN Minimax Objective

Value function:

$$V(D_\psi, G_\theta) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D_\psi(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D_\psi(G_\theta(z)))].$$

Training objective:

$$\min_\theta \max_\psi \; V(D_\psi, G_\theta).$$

Interpretation: D maximizes classification log-likelihood; G tries to minimize it by fooling D.

Non-saturating Generator Loss (practical)

Minimax generator term can saturate if $D(G(z)) \approx 0$. Common alternative (same fixed point, better gradients):

$$\min_{\theta} L_G^{\mathrm{NS}}(\theta) = -\mathbb{E}_{z\sim p_z}\left[\log D_\psi(G_\theta(z))\right].$$

Discriminator still maximizes:

$$\max_{\psi} \ \mathbb{E}_{p_{\text{data}}}[\log D_\psi(x)] + \mathbb{E}_{p_z}[\log(1 - D_\psi(G_\theta(z)))].$$

Step 1: Solve for the Optimal Discriminator $D^\star$

For fixed generator (fixed p_g), maximize over functions $D : \mathcal{X} \to (0, 1)$:

$$V(D, G) = \int p_{\text{data}}(x) \log D(x)\, dx + \int p_g(x) \log(1 - D(x))\, dx.$$

Maximization is pointwise in x. Consider:

$$f_x(D) = p_{\text{data}}(x) \log D + p_g(x) \log(1 - D).$$

Differentiate and set to zero:

$$\frac{d}{dD} f_x(D) = \frac{p_{\text{data}}(x)}{D} - \frac{p_g(x)}{1 - D} = 0 \;\Rightarrow\; D^\star(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Second derivative $-\frac{p_{\text{data}}}{D^2} - \frac{p_g}{(1-D)^2} < 0$ ensures a maximum.

Step 2: Plug $D^\star$ Back (key algebra)

Let $M = \frac{1}{2}(p_{\text{data}} + p_g)$. Substitute $D^\star$:

$$V(D^\star, G) = \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}}{p_{\text{data}} + p_g}\right] + \mathbb{E}_{p_g}\left[\log \frac{p_g}{p_{\text{data}} + p_g}\right]$$
$$= \int p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{2M(x)} dx + \int p_g(x) \log \frac{p_g(x)}{2M(x)} dx.$$

Now split constants:

$$\log \frac{p_{\text{data}}}{2M} = \log \frac{p_{\text{data}}}{M} - \log 2, \qquad \log \frac{p_g}{2M} = \log \frac{p_g}{M} - \log 2.$$

So

$$V(D^\star, G) = -2 \log 2 + \mathrm{KL}(p_{\text{data}} \| M) + \mathrm{KL}(p_g \| M).$$

Step 3: Identify the Jensen–Shannon Divergence

By definition:

$$\mathrm{JS}(p_{\text{data}} \| p_g) = \frac{1}{2}\mathrm{KL}(p_{\text{data}} \| M) + \frac{1}{2}\mathrm{KL}(p_g \| M), \quad M = \frac{p_{\text{data}} + p_g}{2}.$$

Therefore:

$$V(D^\star, G) = -\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \| p_g).$$

Conclusion (theoretical):

$$\min_G \max_D V(D, G) \quad \equiv \quad \min_G \left(-\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \| p_g)\right) \quad \Rightarrow \quad \min_G \mathrm{JS}(p_{\text{data}} \| p_g).$$

Global optimum at $p_g = p_{\text{data}}$, where $\mathrm{JS}(p_{\text{data}} \| p_g) = 0$ and $V = -\log 4$.

What does "GAN decreases JS" actually mean?

Under the *idealized* assumptions:

- discriminator reaches (or tracks) the optimum $D^\star$ for the current G,
- generator can be optimized perfectly,

then minimizing $V(D^\star, G)$ is exactly minimizing $\mathrm{JS}(p_{\text{data}} \| p_g)$. In practice, D is not optimal, optimization is approximate, and the game is non-convex/non-concave, so monotone decrease is *not guaranteed*. Still, the JS connection explains:

- why the discriminator matters (it approximates $D^\star$),
- why support mismatch can cause training pathologies (JS can saturate).

Why Training is Hard: It's a Game, Not a Minimization

GAN training is a saddle-point problem:

$$\min_{\theta} \max_{\psi} \; V(\psi, \theta).$$

Simultaneous gradient descent/ascent can cycle even in simple bilinear games:

$$\min_{\theta} \max_{\psi} \; \psi^\top \theta \quad \Rightarrow \quad \begin{cases} \theta_{t+1} = \theta_t - \eta \psi_t \\ \psi_{t+1} = \psi_t + \eta \theta_t \end{cases} \quad \text{(rotations, not convergence).}$$

With deep nets: nonlinearity + nonconvexity $\Rightarrow$ instability, oscillations, sensitivity to hyperparameters.

Support Mismatch & Gradient Issues (JS saturation intuition)

If p_{data} and p_g lie on low-dimensional manifolds with little overlap, the optimal discriminator becomes near-perfect:

$$D^\star(x) \approx 1 \text{ on data support}, \qquad D^\star(x) \approx 0 \text{ on generator support}.$$

Then the game can provide weak/unstable gradients for G. In particular, JS can saturate near its maximum $\log 2$ when supports are disjoint:

$$\mathrm{JS}(p_{\text{data}} \| p_g) \approx \log 2,$$

and the discriminator can become too confident too quickly.

Mode Collapse: What it is

Mode collapse: p_g puts mass on only a subset of modes of p_{data}.
Extreme collapse (point mass):

$$G_\theta(z) \approx x_0 \quad \forall z \quad \Rightarrow \quad p_g \approx \delta_{x_0}.$$

More generally, collapse to K prototypes:

$$G_\theta(z) \in \{x_1, \ldots, x_K\} \text{ for most } z \quad \Rightarrow \quad p_g \approx \sum_{k=1}^{K} \pi_k \delta_{x_k}.$$

Even if some x_k look realistic, diversity is lost.

Mode Collapse: Discriminator can be fooled locally

Suppose G collapses near a single strong mode of p_{data}. Then D sees many fakes near that mode and might:

- either learn to reject that region (then G may jump to another mode),
- or fail to detect lack of diversity if capacity/optimization is limited.

This yields dynamics like:

$$\text{collapse} \to \text{discriminator adapts} \to \text{generator shifts} \to \text{collapse again.}$$

This can look like *chasing modes* instead of covering them.

More detailed: Why "perfect D" is not always good

If D becomes too accurate early, then for fake samples $x = G(z)$, we can get:

$$D(x) \approx 0.$$

For minimax generator loss:

$$L_G^{\mathrm{MM}} = \mathbb{E}_z[\log(1 - D(G(z)))].$$

Even though $\frac{\mathrm{d}}{\mathrm{d}x} \log(1 - D)$ is finite, the discriminator may be *flat* in regions of fake samples:

$$\nabla_x D(x) \approx 0 \quad \Rightarrow \quad \nabla_\theta L_G \approx 0.$$

This is one reason the non-saturating loss is widely used.

Why Training is Sensitive (practical failure modes)

Common observed issues:

- **Oscillation / non-convergence:** cycling dynamics from game optimization.
- **Discriminator overpowering generator:** D learns too fast $\Rightarrow$ weak G gradients.
- **Mode collapse:** diversity reduction; sometimes sudden.
- **Exploding/vanishing gradients:** due to deep nets + sigmoid saturation + normalization interactions.

Because you're balancing two learners, stability depends on architecture, normalization, learning rates, update ratios, and regularization.

Canonical Training Loop (math + pseudocode)

Algorithm 1 GAN Training (non-saturating G loss)

1: **for** each iteration **do**

2: Sample $\{x_i\}_{i=1}^m \sim p_{\text{data}}$, $\{z_i\}_{i=1}^m \sim p_z$

3: **Discriminator step:**

$$\psi \leftarrow \psi + \eta_D \nabla_\psi \frac{1}{m} \sum_{i=1}^{m} \left[\log D_\psi(x_i) + \log\left(1 - D_\psi(G_\theta(z_i))\right)\right]$$

4: Sample fresh $\{z_i\}_{i=1}^m \sim p_z$

5: **Generator step (NS):**

$$\theta \leftarrow \theta - \eta_G \nabla_\theta \frac{1}{m} \sum_{i=1}^{m} \left[-\log D_\psi(G_\theta(z_i))\right]$$

6: **end for**

Regularizers that directly fight mode collapse

Examples (high-level but math-defined):

- **Feature matching:**

$$\min_{\theta} \|\mathbb{E}_{x\sim p_{\text{data}}}[f_\psi(x)] - \mathbb{E}_{z\sim p_z}[f_\psi(G_\theta(z))]\|_2^2 ,$$

  where $f_\psi(\cdot)$ is an intermediate discriminator feature.

- **Unrolled GAN (idea):** update G accounting for how D will change, approximating a more stable game dynamic.

Why Conditional GANs?

Sometimes we want:

$$x \sim p(x \mid y),$$

where y is a condition (class label, text prompt, attributes, segmentation map, etc.).
Goal: learn a conditional generator:

$$z \sim p_z, \quad x = G_\theta(z, y) \;\Rightarrow\; x \sim p_g(x \mid y).$$

This enables controlled generation: "sample a 7", "generate a face with blond hair", etc.

Conditional GAN Objective (math)

Condition both G and D on y:

$$\min_{\theta} \max_{\psi} \; V(D_\psi, G_\theta) = \mathbb{E}_{(x,y)\sim p_{\text{data}}} \left[\log D_\psi(x, y)\right] + \mathbb{E}_{\substack{y\sim p(y) \\ z\sim p_z}} \left[\log\left(1 - D_\psi(G_\theta(z, y), y)\right)\right].$$

Non-saturating generator loss:

$$\min_{\theta} \; L_G^{\text{NS}}(\theta) = -\mathbb{E}_{y,z} \left[\log D_\psi(G_\theta(z, y), y)\right].$$

If D is optimal *for each condition* y, the same JS story holds condition-wise:

$$\text{idealized:} \;\; \min_{G} \; \mathbb{E}_y \Big[\mathrm{JS}\big(p_{\text{data}}(x|y) \,\|\, p_g(x|y)\big)\Big].$$

How to Implement Conditioning (practical math)

Typical ways to inject y:

- **Concatenate:** $\tilde{z} = [z;\, e(y)]$ then $x = G(\tilde{z})$.
- **Projection discriminator (common in class-conditional GANs):**

$$D(x, y) = u^\top f(x) + \langle v(y), f(x) \rangle,$$

  where $f(x)$ is a feature map and $v(y)$ is a label embedding.

Sampling Conditionally (what you do at inference)

After training, to sample from the conditional model:

$$\text{choose condition } y, \text{ sample } z \sim p_z, \text{ output } \tilde{x} = G_\theta(z, y).$$

For class-conditional generation:

$$y \in \{1, \ldots, C\}, \quad z \sim \mathcal{N}(0, I), \quad \tilde{x} = G(z, y).$$

For text-to-image style conditioning (conceptually):

$$y = \text{text embedding}, \quad \tilde{x} = G(z, y).$$

Key point: conditioning turns GANs from "uncontrolled sampling" to "controlled sampling".

Projection Discriminator: Definition (logit form)

We model the conditional discriminator via a *score* (logit)

$$s_\psi(x, y) \in \mathbb{R}, \qquad D_\psi(x, y) = \sigma\big(s_\psi(x, y)\big).$$

Projection discriminator parameterization:

$$\boxed{s_\psi(x, y) = h_\psi(x) + \langle v(y), f_\psi(x) \rangle}$$

where

$f_\psi(x) \in \mathbb{R}^d$ (image features), $\quad v(y) \in \mathbb{R}^d$ (label embedding), $\quad h_\psi(x) \in \mathbb{R}$ (unconditional score).

Often $h_\psi(x) = u^\top f_\psi(x) + b$, giving

$$s_\psi(x, y) = (u + v(y))^\top f_\psi(x) + b.$$

Key Fact: Optimal discriminator is a density-ratio classifier

Train logistic discriminator with BCE:

$$\max_{\psi} \ \mathbb{E}_{(x,y)\sim p_{\text{data}}} \log D_\psi(x,y) + \mathbb{E}_{(x,y)\sim p_g} \log(1 - D_\psi(x,y)).$$

For *any* fixed generator $p_g(x,y)$, the pointwise optimum is

$$D^\star(x,y) = \frac{p_{\text{data}}(x,y)}{p_{\text{data}}(x,y) + p_g(x,y)}.$$

Therefore the optimal *logit* (inverse-sigmoid) is

$$s^\star(x,y) = \log \frac{D^\star(x,y)}{1 - D^\star(x,y)} = \log \frac{p_{\text{data}}(x,y)}{p_g(x,y)}.$$

If $p_g(y) = p_{\text{data}}(y)$, then

$$s^\star(x,y) = \log \frac{p_{\text{data}}(x \mid y)}{p_g(x \mid y)}.$$

So: learning D is learning a log density-ratio (joint or conditional).

Why the *projection* form matches the ideal logit

We want to approximate

$$s^\star(x, y) = \log \frac{p_{\text{data}}(x, y)}{p_g(x, y)} \quad \text{(a function of both } x \text{ and } y\text{)}.$$

Projection models the (x, y) interaction as a *bilinear* term:

$$s_\psi(x, y) = h_\psi(x) + v(y)^\top f_\psi(x).$$

Think of $f_\psi(x)$ as a learned basis of features and $v(y)$ as label-specific coefficients. Then

$$v(y)^\top f_\psi(x) = \sum_{k=1}^{d} v_k(y)\, f_k(x)$$

selects / reweights features relevant to class y. This is a structured (low-parameter) way to represent a rich family of conditional decision boundaries.

Geometric intuition (one picture in words)

Think of $f(x)$ as mapping images into a feature space where semantics are linear. Each label y defines a direction $v(y)$. Then:

$$v(y)^\top f(x) \text{ is large} \iff f(x) \text{ points toward the label direction.}$$

So the discriminator asks two questions:

$$\underbrace{h(x) \text{ large?}}_{\text{realistic}} \qquad \text{and} \qquad \underbrace{v(y)^\top f(x) \text{ large?}}_{\text{matches label}}.$$

Generator must satisfy both: make realistic images *and* land in the correct label region.

Why not just concatenate y into D? (structural argument)

Concatenation feeds (x, y) and lets the network learn any interaction, but:

- it can ignore y (weak inductive bias),
- the learned interaction may be inefficient (needs more data/parameters),
- gradients may be less label-specific.

Projection *forces* a direct interaction:

$$\boxed{\text{compatibility}(x, y) = v(y)^\top f(x)}$$

which is:

- explicit (cannot be bypassed if used in the logit),
- parameter-sharing across classes (embedding table),
- interpretable as class-specific linear scoring in feature space.

This inductive bias is the main reason it tends to work well.

Implementation formula (clean final form)

Let $f(x) \in \mathbb{R}^d$ be final discriminator features. Let $u \in \mathbb{R}^d$, $b \in \mathbb{R}$. Let $V \in \mathbb{R}^{C \times d}$ be label embeddings.

Score:

$$s(x, y) = u^\top f(x) + b + V_y^\top f(x) = (u + V_y)^\top f(x) + b.$$

Logistic discriminator:

$$D(x, y) = \sigma(s(x, y)).$$

Summary

- VAEs: maximize ELBO

$$\mathcal{L}_{\mathrm{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(z|x) \| p(z)).$$

- GANs: adversarial game

$$\min_G \max_D \ \mathbb{E}_{p_{\text{data}}}[\log D(x)] + \mathbb{E}_{p_z}[\log(1 - D(G(z)))].$$

- With optimal $D^\star$:

$$V(D^\star, G) = -\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \| p_g) \ \Rightarrow \ \text{idealized minimization of JS.}$$

- Limitations: instability (game dynamics), support mismatch, mode collapse.
- Conditional GANs: model $p(x|y)$ via $G(z, y)$ and $D(x, y)$; sample by choosing y then sampling z.

Q&A

Questions?

Tail-GAN: Why a task-specific GAN?

In financial risk management, the goal is not only to generate realistic price paths, but to preserve the *tail risk* of downstream trading strategies.
For a loss variable L, the standard tail-risk measures are

$$\mathrm{VaR}_\alpha(L) = \inf\{\ell : \mathbb{P}(L \le \ell) \ge \alpha\}, \qquad \mathrm{ES}_\alpha(L) = \mathbb{E}[L \mid L \le \mathrm{VaR}_\alpha(L)]\,.$$

Key issue with a vanilla GAN/WGAN objective:

- matching overall sample realism does *not* guarantee accurate tail losses,
- for risk management, rare but severe events matter most,
- so the training loss should emphasize the α-tail, not only the bulk of the distribution.

Core idea: compare scenario distributions through strategies

Let X be a market scenario (a multi-asset price path), and let

$$\Pi = \{\pi_1, \ldots, \pi_K\}$$

be a set of benchmark trading strategies.
Each strategy induces a scalar PnL:

$$L_{\pi_k}(X) \in \mathbb{R}.$$

Tail-GAN does not primarily compare raw price paths. Instead, it compares the *PnL distributions*

$$L_{\pi_k}(X^{\text{real}}) \qquad \text{vs.} \qquad L_{\pi_k}(X^{\text{fake}}), \quad k = 1, \ldots, K.$$

Why this is useful:

- static portfolios probe cross-asset dependence,
- dynamic strategies probe temporal structure,
- matching loss quantiles of many strategies is a financially meaningful proxy for matching the scenario law.

Tail-sensitive discriminator: predict VaR and ES

For each benchmark strategy π_k, given a batch of PnL samples

$$\{L_{\pi_k}(X_i)\}_{i=1}^n,$$

the discriminator outputs estimates of tail-risk statistics:

$$D_\psi(\{L_{\pi_k}(X_i)\}_{i=1}^n) = \left(\widehat{\mathrm{VaR}}_\alpha^{(k)}, \widehat{\mathrm{ES}}_\alpha^{(k)}\right).$$

These predictions are trained with a *strictly consistent score* for the jointly elicitable pair

$$(\mathrm{VaR}_\alpha, \mathrm{ES}_\alpha).$$

So unlike a classical discriminator that says `real/fake`, Tail-GAN's discriminator learns:

- where the tail quantile is,
- how severe the loss is beyond that quantile.

This aligns the adversarial signal directly with tail-risk estimation.

Architecture and training loop

1. Sample noise $z \sim p_z$ and generate price paths:
$$\tilde{X} = G_\theta(z).$$
2. For each benchmark strategy π_k, compute real/fake PnLs:
$$L_{\pi_k}(X), \qquad L_{\pi_k}(\tilde{X}).$$
3. Apply a *differentiable sorting* layer so the network can focus on order statistics relevant to VaR/ES.
4. Discriminator predicts
$$(\widehat{\mathrm{VaR}}_\alpha, \widehat{\mathrm{ES}}_\alpha).$$
5. Update D using the tail-sensitive score; update G through the max–min game so generated scenarios preserve those tail statistics.

Compared with standard GANs, the adversarial game happens in the space of *strategy-induced tail losses*, not just raw samples.

Tail-GAN objective: the tail score for $(\mathrm{VaR}_\alpha, \mathrm{ES}_\alpha)$

For one strategy, let X denote its terminal PnL. The discriminator outputs a pair

$$(v, e) \approx \big(\mathrm{VaR}_\alpha(X), \mathrm{ES}_\alpha(X)\big).$$

Tail-GAN uses a strictly consistent score for the pair $(\mathrm{VaR}_\alpha, \mathrm{ES}_\alpha)$. A general form is

$$S_\alpha(v, e, x) = (\mathbf{1}\{x \le v\} - \alpha)\big(H_1(v) - H_1(x)\big) + \frac{1}{\alpha} H_2'(e)\mathbf{1}\{x \le v\}(v - x) + H_2'(e)(e - v) - H_2(e).$$

The paper uses the quadratic Acerbi–Szekely choice:

$$S_\alpha(v, e, x) = \frac{W_\alpha}{2}\big(\mathbf{1}\{x \le v\} - \alpha\big)(x^2 - v^2) + \mathbf{1}\{x \le v\}\, e(v - x) + \alpha e\Big(\frac{e}{2} - v\Big).$$

So minimizing $\mathbb{E}[S_\alpha(v, e, X)]$ in (v, e) recovers the target

$$(v, e) = \big(\mathrm{VaR}_\alpha(X), \mathrm{ES}_\alpha(X)\big).$$

Sample-based discriminator objective

Let $\Pi^k(p)$ be the PnL of benchmark strategy k on scenario p. Given real scenarios $\{p_i\}_{i=1}^n \sim P_r$, define the PnL sample vector

$$x^k = \big(\Pi^k(p_1), \ldots, \Pi^k(p_n)\big) \in \mathbb{R}^n.$$

The discriminator takes the whole empirical PnL sample as input and outputs

$$D(x^k) = (\hat{v}_k, \hat{e}_k) \in \mathbb{R}^2 \approx \big(\mathrm{VaR}_\alpha, \mathrm{ES}_\alpha\big).$$

Its training objective is

$$D^\star \in \arg\min_{D \in \mathcal{D}} \frac{1}{K} \sum_{k=1}^{K} \frac{1}{n} \sum_{i=1}^{n} S_\alpha\big(D(x^k),\, \Pi^k(p_i)\big).$$

So for each benchmark strategy:

- input = empirical PnL sample of size n,
- output = estimated $(\mathrm{VaR}_\alpha, \mathrm{ES}_\alpha)$,
- loss = average consistent score over the sample.

Generator objective: from bi-level problem to max–min game

Let $\{q_i\}_{i=1}^n \sim P_G$ be generated scenarios and $\{p_i\}_{i=1}^n \sim P_r$ be real scenarios. For strategy k, define

$$x_G^k = (\Pi^k(q_1), \ldots, \Pi^k(q_n)), \qquad x_r^k = (\Pi^k(p_1), \ldots, \Pi^k(p_n)).$$

The ideal bi-level view is:

$$G^\star \in \arg\min_{G \in \mathcal{G}} S_\alpha\big(D^\star(x_G^k),\, \Pi^k(p_j)\big),$$

where $D^\star$ is the optimal discriminator.
In practice the paper uses the equivalent max–min game

$$\max_{D \in \mathcal{D}} \min_{G \in \mathcal{G}} \frac{1}{Kn} \sum_{k=1}^{K} \sum_{j=1}^{n} \Big[S_\alpha\big(D(x_G^k),\, \Pi^k(p_j)\big) - \lambda\, S_\alpha\big(D(x_r^k),\, \Pi^k(p_j)\big) \Big].$$

Reading this objective:

- D wants low score on real samples and high separation from fake ones,
- G wants fake strategy-PnL samples to produce the same tail statistics as real ones,
- the game is played in the space of benchmark-strategy tail losses, not raw paths directly.

Tail-GAN in one sentence

Tail-GAN = GAN training where the critic is designed to care about $(\mathrm{VaR}, \mathrm{ES})$ of strategy PnLs

Takeaways:

- It is a strong example of a **domain-specific GAN objective**.
- The discriminator is no longer a generic real/fake classifier; it is a **tail-risk estimator**.
- Matching raw data distributions is often insufficient when the downstream task is specialized.
- This paper shows how adversarial training can be reshaped around the *actual evaluation metric*.

Why Wasserstein GAN?

Recall the issue with the original GAN objective:

$$V(D^\star, G) = -\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \| p_g).$$

When p_{data} and p_g have little overlap, JS can saturate, so the discriminator becomes nearly perfect and generator gradients can become weak or unstable.
WGAN changes the distance between distributions:

instead of JS / KL-like behavior, use the Wasserstein-1 distance W_1.

Main idea:

- compare distributions by the *cost of transporting mass*,
- obtain a critic objective with more meaningful gradients,
- get a loss that correlates better with sample quality during training.

Wasserstein-1 / Earth-Mover distance

For two distributions P_r and P_g on $\mathcal{X}$, the Wasserstein-1 distance is

$$W_1(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x,y)\sim\gamma}\big[\|x - y\|\big],$$

where $\Pi(P_r, P_g)$ is the set of all couplings with marginals P_r and P_g.
Interpretation:

- $\gamma(x, y)$ says how much probability mass is moved from x to y,
- $\|x - y\|$ is the transport cost,
- W_1 is the minimum total cost needed to morph P_g into P_r.

Why this helps:

$$P_g \to P_r \quad \Longrightarrow \quad W_1(P_r, P_g) \to 0$$

under weaker conditions than for KL / JS, so it behaves better when supports are disjoint or nearly disjoint.

Kantorovich–Rubinstein duality $\Rightarrow$ critic objective

The primal transport problem is hard, but duality gives

$$W_1(P_r, P_g) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_g}[f(x)] \right),$$

where the supremum is over all **1-Lipschitz** functions f.
So instead of a discriminator $D(x) \in (0, 1)$, WGAN uses a **critic**

$$f_w : \mathcal{X} \to \mathbb{R}$$

with no sigmoid output.
If $x = G_\theta(z)$ and $z \sim p(z)$, the WGAN objective becomes

$$\max_{w:\, f_w \in \mathrm{Lip}_1} \mathbb{E}_{x \sim P_r}[f_w(x)] - \mathbb{E}_{z \sim p(z)}[f_w(G_\theta(z))].$$

Generator objective:

$$\min_\theta \mathbb{E}_{z \sim p(z)}[f_w(G_\theta(z))].$$

Why the Lipschitz constraint matters

The dual formula is valid only for 1-Lipschitz critics:

$$|f(x) - f(y)| \leq \|x - y\| \qquad \forall x, y.$$

This constraint prevents the critic from becoming arbitrarily steep and makes it represent a valid Wasserstein dual potential.
Original WGAN enforces this by **weight clipping**:

$$w_i \in [-c, c].$$

Training loop (original WGAN):

- update critic n_{critic} times:

$$\max_w \frac{1}{m} \sum_{i=1}^{m} f_w(x_i) - \frac{1}{m} \sum_{i=1}^{m} f_w(G_\theta(z_i)),$$

  then clip weights;
- update generator once:

$$\min_\theta \frac{1}{m} \sum^{m} f_w(G_\theta(z_i)).$$

WGAN-GP: improved Lipschitz enforcement

Weight clipping is simple but often problematic:

- too small $c \Rightarrow$ underfitting critic,
- too large $c \Rightarrow$ poor Lipschitz control,
- can cause unstable gradients and bad optimization.

A standard improvement is **WGAN-GP**, which adds a gradient penalty:

$$L_D = \mathbb{E}_{\tilde{x}\sim P_g}[f_w(\tilde{x})] - \mathbb{E}_{x\sim P_r}[f_w(x)] + \lambda\, \mathbb{E}_{\hat{x}\sim P_{\hat{x}}}\left(\|\nabla_{\hat{x}} f_w(\hat{x})\|_2 - 1\right)^2,$$

where

$$\hat{x} = \epsilon x + (1-\epsilon)\tilde{x}, \qquad \epsilon \sim \mathrm{Uniform}(0,1).$$

Generator loss:

$$L_G = -\mathbb{E}_{\tilde{x}\sim P_g}[f_w(\tilde{x})].$$

This encourages

$$\|\nabla_{\hat{x}} f_w(\hat{x})\|_2 \approx 1,$$

which is a softer and usually much better way to enforce the Lipschitz condition.

WGAN summary: what changed from vanilla GAN?

Vanilla GAN:

$$\min_G \max_D \; \mathbb{E}_{p_{\text{data}}}[\log D(x)] + \mathbb{E}_{p_z}[\log(1 - D(G(z)))].$$

WGAN:

$$\min_G \max_{f \in \mathrm{Lip}_1} \; \mathbb{E}_{p_{\text{data}}}[f(x)] - \mathbb{E}_{p_z}[f(G(z))].$$

Key differences:

- **critic** $f(x) \in \mathbb{R}$ instead of probability output,
- optimize an approximation to
$$W_1(P_r, P_g),$$
- better behavior under support mismatch,
- training loss is often more meaningful,
- Lipschitz control is essential (clipping in WGAN, gradient penalty in WGAN-GP).